Python Activity-Based Learning

10 Case Studies — Documentation and Python Programs

Student Details

Name	N. Likhitha
Roll Number	26000968
CSE-AIML	
Section	2

1. Bus Passenger Occupancy Simulation

Problem Statement: Track boarding and alighting at multiple stops, calculate occupancy, and flag capacity violations.

Objectives

• Track passengers after every stop.

• Validate counts and prevent impossible alighting.

• Identify occupancy above capacity.

Assumptions

• Bus starts empty.

• Counts are non-negative integers.

• Alighting cannot exceed the current passenger count.

Algorithm

1. Input capacity and number of stops.

2. For each stop, input boarding and alighting.

3. Validate counts and update passenger count.

4. Display occupancy and flag capacity violations.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
INPUT capacity, stops
passengers = 0
FOR each stop:
    INPUT boarding, alighting
    IF boarding < 0 OR alighting < 0 OR alighting > passengers:
        PRINT "Invalid counts"
    ELSE:
        passengers = passengers + boarding - alighting
        PRINT passengers
        IF passengers > capacity: PRINT "CAPACITY VIOLATION"
```

Python Source Code

```
capacity = int(input("Bus capacity: "))
stops = int(input("Number of stops: "))
passengers = 0
for stop in range(1, stops + 1):
    boarding = int(input("Boarding: "))
    alighting = int(input("Alighting: "))
    if boarding < 0 or alighting < 0 or alighting > passengers:
        print("Invalid counts")
        continue
    passengers += boarding - alighting
    print("Passengers after stop", stop, ":", passengers)
    if passengers > capacity:
        print("CAPACITY VIOLATION")
```

Test Cases

Capacity 40; stop data (25,0), (10,5), (15,2). Expected occupancy 25, 30, 43; final stop violates capacity.

Expected Output (illustrative)

```
Passengers after stop 1 : 25
Passengers after stop 2 : 30
Passengers after stop 3 : 43
CAPACITY VIOLATION
```

Conclusion

The simulation calculates occupancy at each stop and identifies when the bus exceeds capacity.

2. Hackathon Team Formation and Ranking

Problem Statement: Validate team sizes, aggregate round scores, and rank teams using tie-break rules.

Objectives

• Validate team size.

• Calculate total and final-round scores.

• Rank by total descending, final-round score descending, then team name ascending.

Assumptions

• Valid team size is 2–4.

• Scores are non-negative.

• Team names are unique.

Algorithm

1. Input team names, sizes, and round scores.

2. Reject invalid sizes or negative scores.

3. Calculate totals and sort accepted teams using the tie-break rules.

4. Display rankings.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each team:
    INPUT name, size, scores
    IF size < 2 OR size > 4 OR any score < 0:
        REJECT team
    ELSE:
        total = SUM(scores)
        SAVE name, total, final score
SORT by total descending, final score descending, name ascending
DISPLAY ranking
```

Python Source Code

```
teams = []
n = int(input("Number of teams: "))
rounds = int(input("Number of rounds: "))
for _ in range(n):
    name = input("Team name: ")
    size = int(input("Team size: "))
    scores = [int(input("Round score: ")) for _ in range(rounds)]
    if not 2 <= size <= 4 or any(x < 0 for x in scores):
        print("Rejected:", name)
    else:
        teams.append((name, sum(scores), scores[-1]))
teams.sort(key=lambda t: (-t[1], -t[2], t[0].lower()))
for rank, team in enumerate(teams, 1):
    print(rank, team[0], "Total:", team[1], "Final round:", team[2])
```

Test Cases

Alpha (3 members; 70,80), Beta (2;75,75), Gamma (5;90,90). Gamma is rejected; Alpha ranks above Beta because final-round score is 80.

Expected Output (illustrative)

```
Rejected: Gamma
1 Alpha Total: 150 Final round: 80
2 Beta Total: 150 Final round: 75
```

Conclusion

The program validates team composition and ranks valid teams according to the scoring and tie-break policy.

3. Food Waste Monitoring System

Problem Statement: Compare meals prepared, served, and leftover across days and flag excessive wastage.

Objectives

• Calculate leftovers.

• Calculate waste percentage.

• Flag days above the accepted limit.

Assumptions

• Served meals cannot exceed prepared meals.

• Counts are non-negative.

• Waste percentage = leftovers / prepared × 100.

Algorithm

1. Input waste limit and number of days.

2. Read prepared and served meals.

3. Validate values and calculate leftovers and percentage.

4. Flag waste above the limit.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
INPUT limit, days
FOR each day:
    INPUT prepared, served
    IF prepared < 0 OR served < 0 OR served > prepared:
        PRINT "Invalid record"
    ELSE:
        leftover = prepared - served
        percent = leftover / prepared * 100 if prepared > 0 else 0
        PRINT leftover, percent
        IF percent > limit: PRINT "LIMIT EXCEEDED"
```

Python Source Code

```
limit = float(input("Acceptable waste percentage: "))
days = int(input("Number of days: "))
for day in range(1, days + 1):
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))
    if prepared < 0 or served < 0 or served > prepared:
        print("Invalid meal record")
        continue
    leftover = prepared - served
    percent = leftover / prepared * 100 if prepared else 0
    print("Leftovers:", leftover, "Waste:", round(percent, 2), "%")
    if percent > limit:
        print("WASTE LIMIT EXCEEDED")
```

Test Cases

Limit 10%; day 1: 100 prepared, 95 served → 5%; day 2: 80 prepared, 60 served → 25% and flag.

Expected Output (illustrative)

```
Leftovers: 5 Waste: 5.0 %
Leftovers: 20 Waste: 25.0 %
WASTE LIMIT EXCEEDED
```

Conclusion

The system measures daily waste and flags days that exceed the acceptable percentage.

4. Retail Order Dispatch and Return Analysis

Problem Statement: Calculate outstanding orders, update inventory after dispatches and returns, and detect inconsistent records.

Objectives

• Calculate outstanding quantity.

• Update closing stock.

• Reject dispatches above orders and returns above dispatches.

Assumptions

• Quantities are non-negative.

• Returned quantity cannot exceed dispatched quantity.

• Returns are added back to stock.

Algorithm

1. Input opening stock, ordered, dispatched, returned.

2. Validate quantities and consistency rules.

3. Calculate outstanding order and closing stock.

4. Print summary.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
INPUT opening, ordered, dispatched, returned
IF any quantity < 0: PRINT invalid
ELSE IF dispatched > ordered OR returned > dispatched:
    PRINT inconsistent record
ELSE:
    outstanding = ordered - dispatched
    closing = opening - dispatched + returned
    DISPLAY outstanding, closing
```

Python Source Code

```
opening = int(input("Opening inventory: "))
ordered = int(input("Quantity ordered: "))
dispatched = int(input("Quantity dispatched: "))
returned = int(input("Quantity returned: "))
values = [opening, ordered, dispatched, returned]
if any(x < 0 for x in values):
    print("Invalid quantities")
elif dispatched > ordered:
    print("Inconsistent: dispatch exceeds order")
elif returned > dispatched:
    print("Inconsistent: returns exceed dispatch")
else:
    print("Outstanding:", ordered - dispatched)
    closing = opening - dispatched + returned
    print("Closing inventory:", closing)
    print("Inventory shortage detected" if closing < 0 else "Inventory valid")
```

Test Cases

Opening 50, ordered 30, dispatched 20, returned 2 → outstanding 10, closing stock 32. Dispatch 35 for order 30 must be rejected.

Expected Output (illustrative)

```
Outstanding: 10
Closing inventory: 32
Inventory valid
```

Conclusion

The program checks record consistency and computes outstanding orders and closing inventory.

5. Energy Conservation Before-and-After Analysis

Problem Statement: Compare appliance energy use before and after an intervention and rank achieved savings.

Objectives

• Calculate kWh savings.

• Calculate savings percentage.

• Rank by absolute savings, then name.

Assumptions

• Usage is non-negative and measured over comparable periods.

• Negative savings mean usage increased.

• If before usage is zero, percentage is reported as 0%.

Algorithm

1. Input appliance name and before/after usage.

2. Validate readings.

3. Calculate savings and percentage.

4. Sort by savings descending and display results.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each appliance:
    INPUT name, before, after
    IF before < 0 OR after < 0: reject
    saved = before - after
    percent = saved / before * 100 if before > 0 else 0
SORT by saved descending, name ascending
DISPLAY results
```

Python Source Code

```
records = []
n = int(input("Number of appliances: "))
for _ in range(n):
    name = input("Appliance: ")
    before = float(input("Before usage (kWh): "))
    after = float(input("After usage (kWh): "))
    if before < 0 or after < 0:
        print("Invalid usage; skipped")
        continue
    saved = before - after
    pct = saved / before * 100 if before else 0
    records.append((name, saved, pct))
records.sort(key=lambda x: (-x[1], x[0].lower()))
for rank, (name, saved, pct) in enumerate(records, 1):
    print(rank, name, "Saved:", round(saved, 2),
          "kWh; savings:", round(pct, 2), "%")
    if saved < 0:
        print("Usage increased")
```

Test Cases

Fan 100→70 kWh saves 30 kWh (30%); AC 300→240 saves 60 kWh (20%). AC ranks first by absolute savings.

Expected Output (illustrative)

```
1 AC Saved: 60.0 kWh; savings: 20.0 %
2 Fan Saved: 30.0 kWh; savings: 30.0 %
```

Conclusion

The program quantifies energy savings and ranks appliances by absolute reduction in usage.

6. Nested List Sports Performance Analysis

Problem Statement: Analyse player scores across matches, calculate average, consistency and trend, and rank players with tie-break rules.

Objectives

• Store match scores in nested lists.

• Calculate average, range, and first-to-last trend.

• Rank by average descending, range ascending, then name.

Assumptions

• Scores are non-negative.

• Each player has at least one score.

• Consistency is represented by score range; lower is more consistent.

Algorithm

1. Input player names and scores across matches.

2. Calculate average, score range and trend.

3. Sort using the stated tie-break rules.

4. Display ranking and metrics.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each player:
    INPUT name, scores
    average = SUM(scores) / length(scores)
    score_range = MAX(scores) - MIN(scores)
    trend = last score - first score
    SAVE record
SORT by average descending, range ascending, name ascending
DISPLAY ranking
```

Python Source Code

```
players = []
n = int(input("Number of players: "))
matches = int(input("Matches per player: "))
for _ in range(n):
    name = input("Player name: ")
    scores = [float(input("Match score: ")) for _ in range(matches)]
    if not scores or any(x < 0 for x in scores):
        print("Invalid scores; player skipped")
        continue
    avg = sum(scores) / len(scores)
    spread = max(scores) - min(scores)
    trend = scores[-1] - scores[0]
    players.append((name, avg, spread, trend))
players.sort(key=lambda x: (-x[1], x[2], x[0].lower()))
for rank, item in enumerate(players, 1):
    print(rank, item[0], "Average:", round(item[1], 2),
          "Range:", round(item[2], 2), "Trend:", round(item[3], 2))
```

Test Cases

Asha [40,50,60]: average 50, range 20, trend +20. Balu [55,50,45]: average 50, range 10, trend –10. Balu ranks first due to lower range.

Expected Output (illustrative)

```
1 Balu Average: 50.0 Range: 10.0 Trend: -10.0
2 Asha Average: 50.0 Range: 20.0 Trend: 20.0
```

Conclusion

Nested lists allow comparison of multiple match scores and provide a consistency-based ranking.

7. Multi-Stage Scholarship Shortlisting

Problem Statement: Apply academic, attendance and income criteria, rank eligible candidates, and select within a fixed quota.

Objectives

• Check all eligibility criteria.

• Rank eligible applicants transparently.

• Do not select more students than the quota.

Assumptions

• Example policy: aggregate ≥75%, each subject ≥40%, attendance ≥75%, income ≤■250,000.

• Ranking: aggregate descending, attendance descending, income ascending, ID ascending.

• Rules are examples and should be replaced with official policy.

Algorithm

1. Input quota and applicant data.

2. Check aggregate, subject marks, attendance and income.

3. Store eligible candidates.

4. Sort and select the first quota candidates.

5. Report selected and eligible-but-not-selected counts.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each applicant:
    INPUT ID, name, aggregate, subject marks, attendance, income
    IF aggregate >= 75 AND all subjects >= 40
       AND attendance >= 75 AND income <= 250000:
        ADD to eligible
SORT by aggregate desc, attendance desc, income asc, ID asc
SELECT first quota candidates
DISPLAY selected list
```

Python Source Code

```
quota = int(input("Scholarship quota: "))
n = int(input("Number of applicants: "))
eligible = []
for _ in range(n):
    sid = input("Student ID: ")
    name = input("Name: ")
    aggregate = float(input("Aggregate percentage: "))
    count = int(input("Number of subjects: "))
    marks = [float(input("Subject marks: ")) for _ in range(count)]
    attendance = float(input("Attendance percentage: "))
    income = float(input("Annual family income: "))
    if aggregate >= 75 and all(m >= 40 for m in marks) and attendance >= 75 and income <= 250000:
        eligible.append((sid, name, aggregate, attendance, income))
eligible.sort(key=lambda s: (-s[2], -s[3], s[4], s[0]))
selected = eligible[:max(0, quota)]
for rank, s in enumerate(selected, 1):
    print(rank, s[0], s[1], "Aggregate:", s[2])
print("Eligible but not selected:", max(0, len(eligible) - len(selected)))
```

Test Cases

Quota 1; S01 aggregate 88 and eligible; S02 aggregate 85 and eligible. S01 is selected; one eligible candidate is not selected.

Expected Output (illustrative)

```
1 S01 Anu Aggregate: 88.0
Eligible but not selected: 1
```

Conclusion

The system applies multiple eligibility stages and selects candidates within the scholarship quota.

8. Integrated University Fee and Eligibility Audit

Problem Statement: Combine fee, attendance and internal marks to determine exam eligibility and list every reason for ineligibility.

Objectives

• Calculate outstanding fee.

• Check attendance and internal marks.

• Report all unmet conditions for each student.

Assumptions

• Example criteria: outstanding fee must be zero, attendance ≥75%, internal marks ≥40/100.

• Paid fees cannot exceed the assessed fee.

• Multiple reasons may be reported.

Algorithm

1. Input student fee, payment, attendance and internal marks.

2. Validate fee data.

3. Calculate balance and collect failed conditions.

4. Print eligibility or a reason-wise report.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each student:
    INPUT ID, assessed_fee, paid, attendance, internal
    IF paid < 0 OR paid > assessed_fee: report invalid data
    ELSE:
        outstanding = assessed_fee - paid
        reasons = []
        IF outstanding > 0: add fee reason
        IF attendance < 75: add attendance reason
        IF internal < 40: add internal reason
        DISPLAY reasons or "Eligible"
```

Python Source Code

```
n = int(input("Number of students: "))
for _ in range(n):
    sid = input("Student ID: ")
    fee = float(input("Total fee assessed: "))
    paid = float(input("Fee paid: "))
    attendance = float(input("Attendance percentage: "))
    internal = float(input("Internal marks out of 100: "))
    if fee < 0 or paid < 0 or paid > fee:
        print(sid, "- Invalid fee record")
        continue
    outstanding = fee - paid
    reasons = []
    if outstanding > 0: reasons.append("Fees outstanding")
    if attendance < 75: reasons.append("Attendance below 75%")
    if internal < 40: reasons.append("Internal marks below 40")
    print("Student:", sid, "Outstanding fee:", round(outstanding, 2))
    print("Not eligible: " + "; ".join(reasons) if reasons else "Eligible for examination")
```

Test Cases

S10: assessed fee ■50,000, paid ■45,000, attendance 70%, internal 35. Expected reasons: outstanding fees, low attendance, low internal marks.

Expected Output (illustrative)

```
Student: S10 Outstanding fee: 5000.0
Not eligible: Fees outstanding; Attendance below 75%; Internal marks below 40
```

Conclusion

The audit produces a reason-wise report from fee and academic records.

9. Multi-Branch Retail Inventory Transfer

Problem Statement: Compare branch stock with target stock and propose transfers while respecting surplus, shortage and per-transfer limits.

Objectives

• Calculate surplus and shortage per branch.

• Transfer stock from donors to receivers.

• Respect maximum transfer size and report remaining shortages.

Assumptions

• Branches stock the same item.

• Stock and target are non-negative integers.

• Transfers have no shipping loss.

• The plan is greedy and follows branch input order.

Algorithm

1. Input each branch's stock and target plus maximum transfer size.

2. Calculate surplus and shortage.

3. Pair receivers with donors and transfer the minimum of surplus, shortage and transfer limit.

4. Report transfers and remaining shortages.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
FOR each branch:
    surplus = max(stock - target, 0)
    shortage = max(target - stock, 0)
FOR each receiver:
    FOR each donor:
        qty = MIN(donor surplus, receiver shortage, max transfer)
        record transfer and update balances
DISPLAY transfers and remaining shortages
```

Python Source Code

```
branches = {}
n = int(input("Number of branches: "))
limit = int(input("Maximum units per transfer: "))
for _ in range(n):
    name = input("Branch name: ")
    stock = int(input("Current stock: "))
    target = int(input("Target stock: "))
    if stock < 0 or target < 0:
        print("Invalid stock; branch skipped")
        continue
    branches[name] = {"surplus": max(stock-target, 0),
                      "shortage": max(target-stock, 0)}
transfers = []
donors = [b for b in branches if branches[b]["surplus"] > 0]
receivers = [b for b in branches if branches[b]["shortage"] > 0]
if limit <= 0:
    print("Transfer limit must be positive")
else:
    for receiver in receivers:
        for donor in donors:
            if donor == receiver: continue
            qty = min(branches[donor]["surplus"],
                      branches[receiver]["shortage"], limit)
            if qty:
                transfers.append((donor, receiver, qty))
                branches[donor]["surplus"] -= qty
                branches[receiver]["shortage"] -= qty
    for donor, receiver, qty in transfers:
        print(donor, "->", receiver, ":", qty, "units")
    for name, data in branches.items():
        if data["shortage"] > 0:
            print("Remaining shortage:", name, data["shortage"])
```

Test Cases

A has stock 100/target 60; B has stock 20/target 50; max transfer 25. Expected transfers A→B 25 and A→B 5.

Expected Output (illustrative)

```
A -> B : 25 units
A -> B : 5 units
```

Conclusion

The program plans stock transfers within the specified limits and reports any remaining shortage.

10. University Hostel Room Allocation

Problem Statement: Allocate students to compatible rooms under capacity and grouping constraints; detect duplicate IDs and count unallocated students.

Objectives

• Respect room capacity and allowed group.

• Avoid assigning a student twice.

• Report duplicate IDs and unallocated demand.

Assumptions

• Each room has a capacity and allowed group.

• Each student has a unique ID and group.

• Rooms are checked in input order; this is a simple greedy method.

Algorithm

1. Input rooms, capacities and allowed groups.

2. Input student IDs and groups.

3. Reject duplicate IDs.

4. Assign each new student to the first compatible room with space.

5. Report room occupancy and unallocated students.

Flowchart (text sequence)

```
[START] -> [INPUT] -> [VALIDATE] -> [PROCESS / CALCULATE] -> [DISPLAY RESULTS] -> [STOP]
```

Pseudocode

```
INPUT rooms and capacities
assigned_ids = empty set
FOR each student:
    IF ID already in assigned_ids: flag duplicate
    ELSE:
        find first compatible room with free capacity
        IF found: assign student
        ELSE: mark unallocated
DISPLAY room assignments, duplicates, unallocated count
```

Python Source Code

```
rooms = []
for _ in range(int(input("Number of rooms: "))):
    name = input("Room name: ")
    capacity = max(0, int(input("Room capacity: ")))
    group = input("Allowed group (A/B): ").upper()
    rooms.append({"name": name, "capacity": capacity,
                  "group": group, "students": []})
assigned_ids, unallocated, duplicates = set(), [], []
for _ in range(int(input("Number of student records: "))):
    sid = input("Student ID: ")
    group = input("Student group (A/B): ").upper()
    if sid in assigned_ids:
        duplicates.append(sid)
        print("Duplicate student ID:", sid)
        continue
    assigned_ids.add(sid)
    for room in rooms:
        if room["group"] == group and len(room["students"]) < room["capacity"]:
            room["students"].append(sid)
            break
    else:
        unallocated.append(sid)
for room in rooms:
    print(room["name"], room["students"], "Occupancy:",
          len(room["students"]), "/", room["capacity"])
print("Duplicate IDs:", duplicates)
print("Unallocated students:", unallocated)
print("Unallocated demand:", len(unallocated))
```

Test Cases

R1 capacity 1 group A; R2 capacity 1 group B. S1/A, S2/A, S3/B, S3/B. S1 in R1, S3 in R2, duplicate S3 flagged, S2 unallocated.

Expected Output (illustrative)

```
R1 ['S1'] Occupancy: 1 / 1
```

```
R2 ['S3'] Occupancy: 1 / 1
Duplicate student ID: S3
Duplicate IDs: ['S3']
Unallocated students: ['S2']
Unallocated demand: 1
```

Conclusion

The allocator respects room capacity and group rules, flags duplicate IDs, and reports unmet accommodation demand.